

LEMOE: LLM-Enhanced Multi-Objective Bayesian Optimization for Microarchitecture Exploration

Jingyuan Li¹, Jianrong Zhang¹, Ye Li², Wenbo Yin¹, Lingli Wang^{1,*}

¹ State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai, China

² School of Computer Science and Engineering, University of New South Wales, New South Wales, Australia

*Email: jingyuanli22@m.fudan.edu.cn, llwang@fudan.edu.cn

Abstract—Designing processor microarchitectures is increasingly challenging due to a vast design space and the need to balance multiple metrics. Traditional algorithm-driven design space exploration (DSE) approaches often struggle to incorporate the extensive domain knowledge of expert architects. To address this, we introduce LEMOE, a multi-objective microarchitecture optimization framework that leverages large language model (LLM) to enhance an implicit Bayesian model. LEMOE features a program-aware warm-up phase utilizing LLM and LLVM to produce an initial design set with rich prior knowledge. By harnessing LLM’s contextual learning, our approach improves surrogate modeling and sampling under sparse data conditions. Experiment results show that LEMOE achieves a 22.8% improvement in energy efficiency with the same number of iterations and a 2.9× runtime speedup for the same target compared to prior works.

Index Terms—In-context learning, Bayesian Optimization, Large Language Model, Microarchitecture design

I. INTRODUCTION

RISC-V, an open-source instruction set architecture (ISA), has become a prominent choice in processor design due to its modular and flexible structure. This adaptability enables extensive customization, making RISC-V popular across diverse applications, from embedded systems [1] [2] and high-performance computing [3] [4] to deep neural networks [5] [6].

Designing a RISC-V microarchitecture is a complex task. The numerous architecture parameters—such as fetch width, branch predictor size, and load/store queue entries—create an expansive combinatorial space. Navigating this complexity requires either experienced designers or efficient design space exploration (DSE) methods to identify configurations that balance multiple objective trade-offs. Microarchitecture evaluation typically relies on commercial or open-source simulators [7], [8], which are both time-consuming and labor-intensive, often taking hours or even days per design evaluation [8]. Therefore, reducing the number of iterations needed to find optimal designs is essential to improve exploration efficiency.

Many methodologies have succeeded in RISC-V DSE in recent years. In [9], bai *et.al* employed a microarchitecture-aware active learning algorithm to generate an initial design set and utilized the Gaussian process with deep kernel learning built to characterize the design space. [10] integrated microarchitecture scaling graphs in reinforcement learning and utilized an agent model to mimic human exploration preference. In

[11] and [12], the authors employed graph representation learning and multi-objective Bayesian optimization with an ensemble model to explore microarchitecture within a compact embedding space. In [13], a co-training k-nearest neighbors model was used to fit microarchitecture features and assist in searching for a Pareto frontier through parallel emulation.

Existing optimization methods can be broadly categorized into black-box and white-box approaches. Black-box optimization [9] [11] [13] [14] [15] is used in scenarios where the internal structure of the system is completely unknown. This approach relies on observing inputs and outputs, as the objective function’s internal mechanisms are inaccessible. While it is more adaptable to different architectures, it often involves blindly testing inputs to identify optimal solutions. Additionally, it also struggles to incorporate designers’ prior knowledge, potentially overlooking key design features that architects are already familiar with. White-box optimization [10] [16] [17], on the other hand, leverages knowledge of the system’s internal structure. Although white-box approaches can be highly efficient, developing such frameworks requires experienced designers and often lacks portability when the target architecture changes.

Large language models (LLMs) offer a promising bridge between black-box and white-box optimization approaches. Models like GPT-4 [18] and BERT [19] have rapidly evolved into transformative technologies in artificial intelligence. Trained on vast amounts of textual data, these models learn to understand and generate text by identifying complex linguistic patterns and contextual relationships. This capability enables LLMs to produce coherent responses through in-context learning [20]. Recent studies have demonstrated the potential of LLMs in optimization tasks. For instance, [21] and [22] explored the use of LLMs to implement genetic search operators in evolutionary algorithms, while [23] validated the feasibility of using LLMs as surrogate models in Bayesian optimization. The extensive RISC-V domain knowledge embedded in LLMs’ training corpora inspires us to integrate the human expert prior knowledge into traditional black-box optimization process.

Specifically, this article presents LEMOE, a novel multi-objective framework for RISC-V microarchitecture DSE. LEMOE introduces a program-aware initialization approach leveraging LLVM. By analyzing input programs at a fine-grained level using LLVM, the framework utilizes prior knowl-

*Corresponding author.

edge of RISC-V designs and program features for a warm-start. In the Bayesian optimization, LEMOE replaces the non-informative Gaussian process, integrating an LLM as a surrogate model. This approach combines the strengths of black-box and white-box optimization, utilizing the LLM’s rich prior knowledge and designing an insightful sampler to enhance exploration efficiency. Our contributions are summarized as follows:

- We introduce a program-aware initialization method for warm-starting the exploration process. By leveraging the prior knowledge of LLM in RISC-V and LLVM feature analysis, this approach effectively guides iterative exploration.
- We propose a novel multi-objective optimization algorithm that leverages an LLM as a surrogate model and incorporates an insightful sampler for implicit Bayesian inference. This method facilitates efficient trade-offs between performance and power while mitigating the inefficiencies of black-box optimization.
- We perform an ablative study on the various modules and parameters of LEMOE. Comparative analyses with previous work highlight the advantages of LEMOE in enhancing multi-objective exploration efficiency. To the best of our knowledge, this is the first study to utilize an LLM to improve multi-objective Bayesian optimization for RISC-V microarchitecture exploration.

Paper Organization: Section II provides an overview of the background and essential preliminaries. Section III presents our microarchitecture exploration framework. Section IV compares with previous works and performs ablation experiments. Section V comprises the conclusion.

II. BACKGROUND & PRELIMINARIES

A. Microarchitecture modeling

We utilize Chipyard [24] to model our RISC-V microarchitecture. Chipyard is an open-source framework for designing, simulating, and deploying Chisel-based hardware systems. The Berkeley Out-of-Order Machine (BOOM) is a high-performance RISC-V core that implements out-of-order execution. BOOM is designed to be modular and configurable, offering a wide array of tunable parameters. Key architectural features such as issue width, pipeline depth, cache hierarchy, and branch prediction mechanisms can all be adjusted to meet specific design goals. However, this flexibility comes with the challenge of managing a complex design space, as the interactions between these parameters are often non-linear and difficult to predict. This makes it important to explore the design space to find configurations that balance multiple objective trade-offs. The design space of the BOOM we target is shown in TABLE I. The definition of these parameters can refer to [25].

B. Definition & Target

Pareto frontier. Pareto frontier is a set of optimal solutions in multi-objective optimization where improving one objective means sacrificing another. For a M -dimension minimization

TABLE I: DESIGN SPACE OVERVIEW

Module	Parameter	Value
I-cache	numSets	1, 4, 8
	numWays	32, 64
D-cache	numSets	64, 128
	numWays	2, 4, 8
	numMSHR	2, 4, 8
IDU	decodeWidth	1, 2, 3, 4, 5
	numRobEntries	30, 36, 60, 72, 90, 108, 136
	IntPhyRegister	42, 62, 70, 80, 90, 100, 110, 128, 146
	FpPhyRegister	38, 48, 54, 64, 74, 86, 96, 118, 128
EU	memIssueWidth	1, 2
	intIssueWidth	1, 2, 3, 4, 5
	fpIssueWidth	1, 2, 3, 4, 5
LSU	numMemEntries	6, 10, 14, 20, 28
	numLdqEntries	6, 8, 12, 16, 20, 24, 28, 32
IFU	fetchWidth	4, 8
	FetchBufferEntries	6, 12, 16, 24, 30, 36, 45
Branch	maxBrCount	6, 8, 12, 16, 20, 24
	FetchTargetQueue	14, 20, 30, 36, 50

problem, for two solutions $s_i = (f_1^i, \dots, f_M^i)$ and $s_j = (f_1^j, \dots, f_M^j)$, s_i dominates s_j ($s_i \succeq s_j$) if and only if $f_1^i \leq f_1^j, f_2^i \leq f_2^j, \dots, f_M^i \leq f_M^j$. s_i is *Pareto efficient* if no other solutions dominate s_i . All such Pareto efficient solutions are defined as the *Pareto frontier*.

Hypervolume. Assume a set of sampling points after evaluation $\mathcal{S} = \{s_1, \dots, s_n\}$, given its Pareto frontier, $\mathcal{P}(\mathcal{S})$. Choose a reference point $s_{ref} \in \mathbb{R}^L$, which is dominated by each point in $\mathcal{P}(\mathcal{S})$. The hypervolume is calculated as:

$$HV(\mathcal{P}(\mathcal{S})) = \int_{\mathbb{R}^L} \mathbb{I}[s \succeq s_{ref}] [1 - \prod_{s^* \in \mathcal{P}(\mathcal{S})} \mathbb{I}[s^* \not\succeq s]] ds \quad (1)$$

where $\mathbb{I}(\cdot)$ is the indicator function, which outputs 1 if its argument is true and 0 otherwise. A higher hypervolume indicates a better spread and coverage of optimal solutions, showing a balanced trade-off across the objectives.

Target. The BOOM microarchitecture is evaluated based on two key metrics: performance, measured as *instructions per cycle (IPC)*, and *power*. High-performance designs often lead to increased power consumption, our goal is to identify Pareto-efficient microarchitecture designs that effectively balance these metrics within a limited number of iterations.

III. LEMOE FRAMEWORK

A. Overview

The LEMOE framework is outlined in Fig. 1 and comprises five steps. **Program-Aware Initialization:** LLVM analyzes the program set, generating features that are transformed into an LLM prompt to initialize Bayesian optimization. **Sampling:** Both existing data and the LLM’s prior knowledge of the BOOM microarchitecture, obtained through in-context learning, are used for sampling. **Surrogate Modeling:** The LLM serves as a surrogate model to estimate the sampled points. **Acquisition Function:** A multi-objective acquisition function is introduced to identify optimal sampling points. **RISC-V Evaluation:** The selected sampling points undergo evaluation

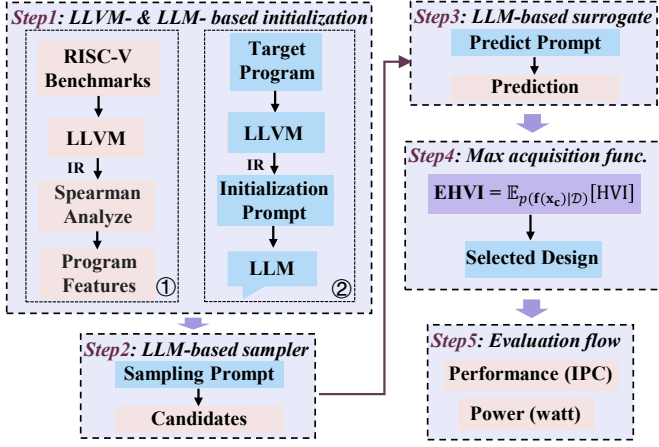


Fig. 1: LEMOE flowchart overview.

TABLE II: PROGRAM FRATURES

Type	Feature	Description
Structure	numBB	# of basic blocks
	avgPred	Average predecessors of a BB
	avgSucc	Average successors of a BB
	numBranch	# of branch instructions
	numInst	# of instructions (of all types)
	numCfgEdges	# of edges in CFG ¹
	numPhi	# of <i>Phi</i> nodes
	numRet	# of return instructions
Memory	numCall	# of call instructions
	numLoad	# of load instructions
	numStore	# of store instructions
Computation	numAlloc	# of alloca instructions
	numUnaryInst	# of unary instructions
	numBinalnst	# of binary instructions
	numComp	# of computation instructions
	numFloatComp	# of float instructions

¹ : CFG represents Control Flow Graph.

within the VLSI flow. Steps 2 to 5 are iteratively repeated to identify the final Pareto frontier.

B. Program-aware Initialization

Compared to random initialization [26] or initialization based on architecture features [9], program-aware initialization methods aim to capture the structure of the target program efficiently. Input program features directly influence design decisions related to performance optimization, energy efficiency, and resource allocation. For instance, if a program exhibits many memory-bound operations, optimizing the cache hierarchy—such as adjusting cache associativity or block size—can significantly enhance performance. Similarly, frequent floating-point operations in a program may warrant optimized issue widths for floating-point instructions. To achieve this, input programs written in C/C++ are processed through an LLVM front-end, which translates the source code into LLVM Intermediate Representation (IR). The LLVM IR retains detailed information about the program’s control flow, data types, and instructions. A summary of the program’s parsed characteristics is provided in TABLE II.

You are assisting me with automated RISC-V BOOM core parameters setting, the following are paremeters and the range of their values: {design space}. The average characteristics of the input program are: {program features}. Please suggest {num} diverse yet effective configurations based on the input program characteristics to initiate a Bayesian Optimization process. You mustn't include "None" in the configurations. Your response should include only a list of dictionaries, where each dictionary describes one recommended configuration. Do not enumerate the dictionaries.

Fig. 2: Program-aware initialization prompt. Blue text indicates variables.

TABLE III: FEATURE SELECTION

Key Feature	Highly correlated	Correlation
numBB	numBranch	0.99
	numCfgEdges	0.99
	numPhi	0.95
	numCall	0.96
numLoad	numStore	0.99
numInst	numUnaryInst	0.91
	numBinalnst	0.93
	numComp	0.98
avgPred	avgSucc	0.98

However, the features in TABLE II are not entirely independent. For instance, the number of basic blocks (*numBB*) is closely related to the number of branch instructions (*numBranch*), as the structure and quantity of basic blocks significantly influence the number of branch instructions. To streamline program feature information, we extract features from the target benchmark set [27] and analyze the Spearman correlation coefficient [28] between them. This coefficient ranges from -1 to 1, with values closer to -1 or 1 indicating a stronger correlation between the two features. Finally, we remove the features with the correlation absolute value greater than 0.9, as shown in TABLE III, and select the following 6 features: *numBB*, *numLoad*, *numInst*, *numRet*, *avgPred*, *numFloatComp*. Then we initialize the design set using the prompt as shown in Fig. 2.

Program-aware initialization leverages prior knowledge of specific workloads to guide the selection of initial design parameters. Aligning initialization with program characteristics ensures that exploration begins with optimized and relevant configurations, avoiding the selection of highly implausible initial points. This methodology makes microarchitecture exploration application-specific. The evaluation of this approach is detailed in Section IV.

C. LLM-enhanced Bayesian optimization

After the initialization phase, we proceed by iteratively selecting the most promising points for exploration. Consider the sampled dataset $\mathcal{D}_n = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ comprising n sets of microarchitecture design $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^n$ and m corresponding evaluation metrics $\mathcal{Y} = \{\mathbf{y}_i\}_{i=1}^n = \{y_1^1, \dots, y_n^1; y_1^2, \dots, y_n^2; \dots; y_1^m, \dots, y_n^m\}^\top$, the relationship be-

```

The following outlines the parameter configurations for
a RISC-V BOOM core, along with the corresponding metrics
measured in [IPC, power]. The BOOM core consists of
{tot_feats} parameters, with allowable ranges defined as
{design space}. Recommend a configuration that achieves
the target power-performance hypervolume of {target}.
Avoid selecting values at the minimum or maximum of the
allowable range. Your response should only include the
predicted configuration in the format
## configuration ##.
Metric: {IPC1,power1}
Configuration: {configuration 1}
...
Metric: {IPCN,powerN}
Configuration: {configuration N}
Metric:{target}
Configuration:

```

Fig. 3: Sampling prompt.

```

The following outlines the parameter configurations for
a RISC-V BOOM core, along with the corresponding metric
measured in [IPC, power]. The BOOM core consists of
{tot_feats} parameters. Your response should only
include the predicted [IPC, power] in the format
## [IPC, power] ##.
Configuration: {configuration 1}
Metric: {IPC1,power1}
...
Configuration: {configuration N}
Metric: {IPCN,powerN}
Configuration: {configuration to predict}
Metric:

```

Fig. 4: Surrogate model prompt.

tween the samples is independently and identically distributed. When considering a new sampling point x^* , the predictive distribution of the d -th objective can be given by:

$$p(y^d|x^*; \mathcal{D}_n) = \int_{\Theta} p(y^d|x^*; \mathcal{D}_n, \theta) p(\theta|x^*; \mathcal{D}_n) d\theta \quad (2)$$

where θ is a latent variable and $p(\theta|x^*; \mathcal{D}_n)$ denotes the posterior distribution. In Bayesian optimization, a Gaussian process (GP) is often used as a surrogate model to capture the relationship between x^* and y^d , where θ represents the hyperparameters of the GP model. From the Bayes' theorem, we note that $p(\theta|x^*; \mathcal{D}_n) \propto p(\theta)p(\mathcal{D}_n|\theta, x^*)$, $p(\theta)$ serves an important prior role. However, traditional black-box optimization often relies on an uninformative prior, potentially overlooking valuable domain-specific knowledge that could enhance optimization efficiency. This limitation inspired us to leverage LLMs to enhance Bayesian optimization.

An effective surrogate model should accurately represent the central predicted values for different input points, along with the confidence or uncertainty in those predictions to balance between exploration and exploitation. For this purpose, we represent the sampled dataset $\mathcal{D}_n$ as a natural text $\mathcal{D}'_n$, which is combined with a predefined prompt to generate mean and uncertainty of x^* for estimation, denoted as $(m^d, v^d) = LLM(\mathcal{D}'_n, x^*)$. To increase robustness, we

introduce a shuffling mechanism in $\mathcal{D}'_n$, since LLM is sensitive to the order of examples in the context [29].

The acquisition function uses the predictions of the surrogate model to determine where to sample next. Common acquisition functions [30] are probability of improvement (PI), expected improvement (EI), and upper confidence bound (UCB). To better balance power and performance, we introduce expectation hypervolume improvement (EHVI) [31] to evaluate the final candidate solutions and it is defined as follows:

$$EHVI(x_{n+1}|\mathcal{D}) = \mathbb{E}_{p(f(x_{n+1})|\mathcal{D})}[HV_{n+1} - HV_n] \quad (3)$$

Typically, HV_{n+1} is calculated using the posterior distribution of x_{n+1} generated by the surrogate model. We introduce the LLM-based sampling method to compute EHVI, which offers three main advantages. First, the EHVI function is notorious for its high computational complexity, and the LLM-based non-analytical method reduces the computational overhead. Second, explicit uncertainty estimation for the sampling points is unnecessary with the non-analytical method. The LLM only needs to estimate the mean values of the sampling points, denoted as $m^d = LLM(\mathcal{D}'_n, x^*)$. This is a method similar to the popular SMAC [32] for obtaining probability estimates. Third, the LLM-based sampler incorporates prior knowledge of RISC-V microarchitecture, which enhances sampling efficiency. Comparative experiments will be presented in Section IV.

To sample design points that would improve exploration, we set target values for exploration. For multi-objective optimization, we use hypervolume as a composite metric due to its comprehensive response to solution quality. The following objective is used as sampling expectations:

$$hv^* = hv_{cur} + \alpha \times (hv_{cur} - hv_{init}) \quad (4)$$

where hv_{cur} and hv_{init} denote the currently realized hypervolume and the hypervolume of the initial sampling set. hv^* is defined relative to the best hypervolume value, with the difference proportional to the observed improvement in hypervolume. The hyperparameter $\alpha > 0$ regulates explorability, where larger values of α result in higher expectations. The determination of the parameters is studied by Section IV. We use the prompt shown in Fig.3 to sample N candidate points in parallel, denoted as $\mathcal{C} = \{x_n\}_{n=1}^N$. These points are then evaluated using the LLM to estimate performance and power, as illustrated in Fig.4, resulting in $\hat{\mathcal{M}} = \{y_n\}_{n=1}^N$. After collecting and estimating N points in parallel during each iteration, the next potential point x^* is sampled by maximizing the EHVI:

$$x^* = \arg \max_{x \in \mathcal{C}} [EHVI(\mathcal{D}_n, \mathcal{C}, \hat{\mathcal{M}})] \quad (5)$$

The design x^* is then fed into the automated RISC-V modeling and evaluation flow. The pseudocode of our framework is presented in Algorithm 1.

Algorithm 1 LEMOE pseudocode

Input:

$\mathcal{X}$ the design space of RISC-V microarchitecture;
 $\mathcal{B}$ target program set;
 $N_i, N_l, N \in \mathbb{N}$: numbers of initial samples & LLM
samples per iteration & iterations;

Output:

Pareto-optimal design $\mathcal{P}$;

```
1:  $\mathcal{I} \leftarrow \text{ProgramAwareInit}(N_i, \mathcal{X}, \mathcal{B})$ ;  
2:  $\mathcal{D} = \text{Evaluate}(\mathcal{I}) \cup \mathcal{I}$ ;  
3: for  $t \leftarrow 1, 2, \dots, n$  do  
4:    $\mathcal{C} \leftarrow \text{LLMSampler}(\mathcal{D}, N_l)$ ;  
5:    $\{(i_l, p_l)\}_{l=1}^{N_l} \leftarrow \text{LLMSurrogate}(\mathcal{C})$ ;  
6:    $x^* \leftarrow \arg \max_{x \in \mathcal{C}} [\text{EHVI}(\mathcal{D}, \mathcal{C}, \{(i_l, p_l)\}_{l=1}^{N_l})]$ ;  
7:    $\mathcal{D} = \text{Evaluate}(x^*) \cup \mathcal{D}$ ;  
8: end for  
9: return  $\mathcal{P} = \text{Pareto}(\mathcal{D})$ 
```

IV. EVALUATION

In this section, we systematically evaluate the performance of LEMOE compared to previous works, conduct ablative studies, and provide quantitative and qualitative analyses.

A. Experiment setup

We compare LEMOE with several prior works, including the Random Forest algorithm, the Tree-structured Parzen Estimator (TPE) algorithm [33] (NeurIPS'11), its multi-objective version [34] (JAIR'22), the ML-based architecture exploration platform [35] (ISCA'23), BOOM-Explorer [9] based on Bayesian optimization and deep kernel learning (ICCAD'21), and a single-objective optimization framework using LLM sampling [36] (ICLR'24).

In our framework, we use GPT-3.5 and GPT-4o as the LLMs. The total number of sampling points is set to 15, with an initial sample size of 5 points. Our method utilizes a program-aware initialization, while the ICCAD'21 method employs a microarchitecture-aware active learning algorithm, and the other methods use random initialization. The test benchmarks, sourced from [27], include *mm*, *median*, *vvadd*, and *qsort*. These benchmarks cover all types of instructions. Metrics are evaluated based on the average power and average IPC across all four benchmarks. Experiments are conducted on a workstation with 2 Intel Xeon 2.10GHz CPUs and 256 GB memory.

B. Initialization comparison

We begin with a systematic study of the individual components of LEMOE before performing an end-to-end comparison. Specifically, we compare different initialization methods: random initialization, Latin Hypercube Sampling (LHS), initialization using only the LLM, and the program-aware initialization based on LLM and LLVM. For each method, we calculate the Spearman correlation matrix and the average

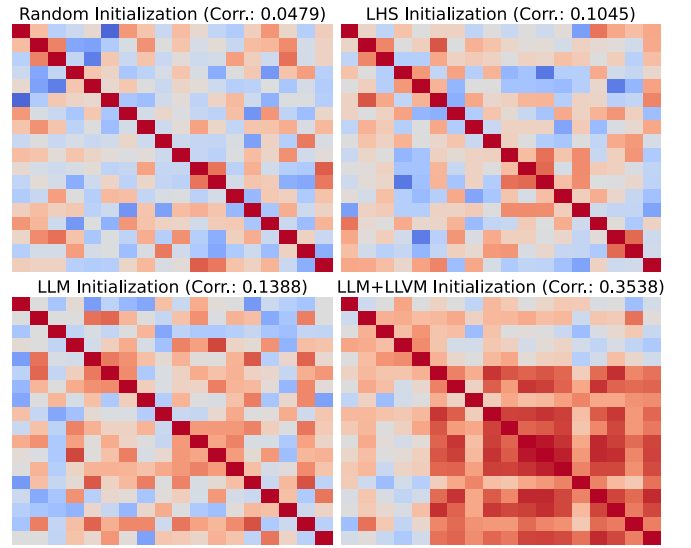


Fig. 5: Correlation heatmap of sampled points. The top shows the average correlation for each method.

correlation for 50 initial sampling points. The correlation heatmap is shown in Fig. 5.

The top of the figure illustrates the average correlation for each method. The results show that the LLM-based initialization method achieves a higher correlation between sampling point parameters compared to both the random initialization and LHS methods. This indicates that LLM effectively captures parameter relationships. Moreover, the LLVM- and LLM-based program-aware initialization further enhances sampling correlations, offering greater insight into the input program compared to the LLM-based method.

C. Ablation study

We conduct an end-to-end comparison of LEMOE using three configurations: (L1) using random sampling without the LLM-based sampler, (L2) omitting LLVM IR for initialization, (L3) excluding RISC-V-specific content for contextual learning, and finally, the full LEMOE configuration (L4). We evaluate the hypervolume and IPC/power under GPT-3.5 and GPT-4o, as shown in TABLE IV. The results show that using random sampling instead of LLM reduces solution quality due to the absence of an insightful sampler. Similarly, omitting LLVM for feature enhancement results in a decline in quality. The most significant quality drop occurs when RISC-V-specific information is excluded, highlighting the crucial role of prior knowledge in an efficient exploration framework.

In equation 4, α represents the intensity of our exploration, with larger values of α indicating more aggressive exploration. We evaluate different values of α under GPT-3.5 and GPT-4o, as shown in TABLE V. With GPT-3.5, an α of 0.05 yields the highest solution quality, while with GPT-4o, a more aggressive α of 0.1 performs best. Both larger (0.2) and smaller (0.01) α values tend to reduce solution quality.

TABLE IV: ABLATION STUDY

LLM	Method	IPC / Power ¹	Ratio	HV ²	Ratio
GPT-3.5	L1	42.271	1.213×	0.754	85.6%
	L2	49.107	1.409×	0.824	93.5%
	L3	34.853	1	0.676	76.7%
	L4	51.369	1.474×	0.881	1
GPT-4o	L1	43.473	1.078×	0.801	89.6%
	L2	49.196	1.220×	0.854	95.4%
	L3	40.321	1	0.752	84.0%
	L4	52.090	1.292×	0.895	1

¹ : Power is measured in milliwatts (mW).

² : HV represents hypervolume.

TABLE V: IMPLICATIONS OF EXTRAPOLATION

LLM	Alpha	IPC / Power	Ratio	HV	Ratio
GPT-3.5	0.2	47.931	1.092×	0.823	93.4%
	0.1	46.241	1.054×	0.845	95.9%
	0.05	51.369	1.170×	0.881	1
	0.01	43.891	1	0.820	93.0%
GPT-4o	0.2	48.661	1.112×	0.847	94.6%
	0.1	52.090	1.191×	0.895	1
	0.05	48.197	1.102×	0.840	93.8%
	0.01	43.753	1	0.816	91.2%

D. End-to-end Comparison

The results of end-to-end comparison with prior works are shown in TABLE VI. Among all methods, our framework utilizing GPT-4o achieves superior performance. Compared to Ours-4o, the results show a 45.3% improvement in the IPC/Power and a 22.1% increase in the hypervolume over the Random Forest. Relative to NeurIPS'11 and JAIR'22, IPC/power improves by 23.3% and 7.5%, while hypervolume improves by 24.5% and 16.0%, respectively. For ISCA'23, improvements of 11.6% and 12.9% are observed, while comparisons with ICLR'24 and ICCAD'21, our method shows respective improvements of 19.2% and 29.6% in IPC/power and 17.2% and 9.0% in hypervolume. Figure 6 illustrates the Pareto frontiers achieved by each method. For clearer visualization, we use the inverse of IPC, represented as cycles per instruction (CPI), ensuring that both metrics are minimized.

We also compare runtime, which consists of two parts: the surrogate modeling time, typically very short (ranging from seconds to a few minutes), and the EDA tool's runtime, which takes several hours to tens of hours per design evaluation. Consequently, the surrogate modeling time constitutes

TABLE VI: COMPARISON WITH PRIOR WORKS

Method	IPC / Power	Ratio	HV	Ratio	RHT ¹
Random Forest	35.846	1	0.697	77.9%	3.509×
NeurIPS'11 [33]	42.263	1.179×	0.676	75.5%	3.565×
JAIR'22 [34]	48.460	1.351×	0.751	84.0%	2.925×
ISCA'23 [35]	46.676	1.302×	0.780	87.1%	2.167×
ICCAD'21 [9]	40.178	1.121×	0.814	91.0%	1.956×
ICLR'24 [36]	43.695	1.219×	0.741	82.8%	2.996×
Ours-3.5	51.369	1.433×	0.881	98.5%	-
Ours-4o	52.090	1.453×	0.895	1	-

¹ : RHT represents Runtime ratios for the same Hypervolume Target.

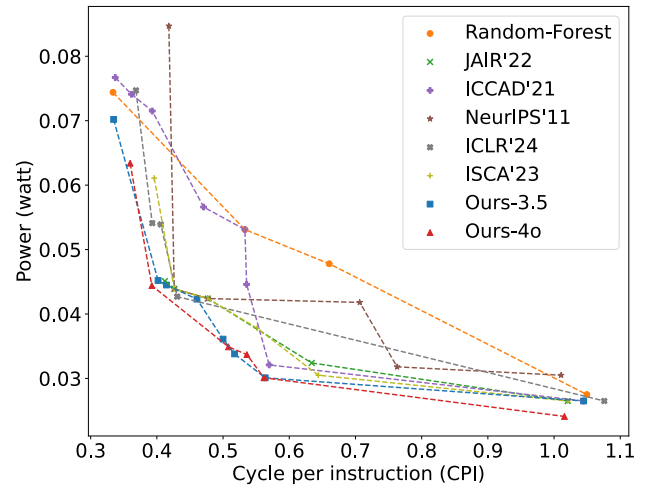


Fig. 6: Pareto frontiers of different methods, where a smaller indicator represents better quality. Note that CPI and IPC are inversely related.

a minimal portion of the total runtime. The previous works perform modeling locally, while the LLM operates in the cloud. Although this setup makes the runtime comparison somewhat lenient, the impact of this difference is negligible. We evaluate the Runtime ratio for the same Hypervolume Target (RHT), which compares the total runtime of prior methods to the runtime required by our Ours-4o method to achieve the same hypervolume. Higher RHT values indicate faster runtime for reaching the same target. As shown in TABLE VI, our method achieves a speedup ranging from 1.956×

V. CONCLUSION

This article presents LEMOE, a multi-objective optimization framework that leverages the LLM-enhanced Bayesian model to explore the design space of RISC-V microarchitecture. LEMOE incorporates domain-specific knowledge through LLM integration and employs a program-aware initialization algorithm, utilizing LLVM and LLM to warm up the optimization effectively. By replacing the traditional Gaussian process with LLM and incorporating a multi-objective acquisition function for sampling and prediction, LEMOE enhances iterative efficiency. Evaluation of LEMOE on the BOOM demonstrates its capability to optimize key metrics. Future work includes comparing additional LLMs and addressing potential hallucination phenomena that may arise with LLMs.

ACKNOWLEDGMENT

This work is supported by National Science and Technology Major Project (2021ZD0114701).

REFERENCES

- [1] B. Sá, J. Martins, and S. Pinto, "A first look at risc-v virtualization from an embedded systems perspective," *IEEE Transactions on Computers*, vol. 71, no. 9, pp. 2177–2190, 2021.
- [2] Y. Kra, Y. Shoshan, Y. Rudin, and A. Teman, "Hamsa-di: A low-power dual-issue risc-v core targeting energy-efficient embedded systems," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2023.
- [3] F. Minervini, O. Palomar, O. Unsal, E. Reggiani, J. Quiroga, J. Marimon, C. Rojas, R. Figueras, A. Ruiz, A. Gonzalez, *et al.*, "Vitruvius+: an area-efficient risc-v decoupled vector coprocessor for high performance computing applications," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 2, pp. 1–25, 2023.
- [4] A. Bartolini, F. Ficarelli, E. Parisi, F. Beneventi, F. Barchi, D. Gregori, F. Magugliani, M. Cicala, C. Gianfreda, D. Cesarini, *et al.*, "Monte cimone: paving the road for the first generation of risc-v high-performance computers," in *2022 IEEE 35th International System-on-Chip Conference (SOCC)*, pp. 1–6, IEEE, 2022.
- [5] S. Lee, J. Choi, S. Noh, J. Koo, and J. Kung, "Dbps: Dynamic block size and precision scaling for efficient dnn training supported by risc-v isa extensions," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2023.
- [6] L. Huang, C. Fang, Q. Li, J. Lin, and Z. Wang, "A precision-scalable risc-v dnn processor with on-device learning capability at the extreme edge," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 927–932, IEEE, 2024.
- [7] J. Zhai, C. Bai, B. Zhu, Y. Cai, Q. Zhou, and B. Yu, "Mcpat-calib: A risc-v boom microarchitecture power modeling framework," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 1, pp. 243–256, 2022.
- [8] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, *et al.*, "The gem5 simulator," *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.
- [9] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, "Boom-explorer: Risc-v boom microarchitecture design space exploration framework," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pp. 1–9, IEEE, 2021.
- [10] C. Bai, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, "Towards automated risc-v microarchitecture design with reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 12–20, 2024.
- [11] X. Yi, J. Lu, X. Xiong, D. Xu, L. Shang, and F. Yang, "Graph representation learning for microarchitecture design space exploration," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2023.
- [12] Z. Wu, J. Shen, X. Yi, L. Shang, F. Yang, and X. Zeng, "Prior-boosted grl: Microarchitecture design space exploration via graph representation learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [13] X. Zheng, M. Cheng, J. Chen, H. Gao, X. Xiong, and S. Cai, "Bsse: Design space exploration on the boom with semi-supervised learning," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2024.
- [14] D. Luo, Q. Sun, X. Li, C. Bai, B. Yu, and H. Geng, "Knowing the spec to explore the design via transformed bayesian optimization," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pp. 1–6, 2024.
- [15] J. Li, Y. Hu, Y. Dai, H. Kuang, and L. Wang, "Auger: A multi-objective design space exploration framework for cgras," in *2023 International Conference on Field Programmable Technology (ICFPT)*, pp. 88–95, IEEE, 2023.
- [16] B. Sá, L. Valente, J. Martins, D. Rossi, L. Benini, and S. Pinto, "Cva6 risc-v virtualization: Architecture, microarchitecture, and design space exploration," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.
- [17] N. A. Koca, C. H. Chang, A. T. Do, and V. P. Nambiar, "Exploring error correction circuits on risc-v based systems for space applications," in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, IEEE, 2024.
- [18] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [19] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [20] Q. Dong, L. Li, D. Dai, C. Zheng, Z. Wu, B. Chang, X. Sun, J. Xu, and Z. Sui, "A survey on in-context learning," *arXiv preprint arXiv:2301.00234*, 2022.
- [21] A. Chen, D. Dohan, and D. So, "Evoprompting: language models for code-level neural architecture search," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [22] J. Lehman, J. Gordon, S. Jain, K. Ndousse, C. Yeh, and K. O. Stanley, "Evolution through large models," in *Handbook of Evolutionary Machine Learning*, pp. 331–366, Springer, 2023.
- [23] M. C. Ramos, S. S. Michtavy, M. D. Porosoff, and A. D. White, "Bayesian optimization of catalysts with in-context learning," *arXiv preprint arXiv:2304.05341*, 2023.
- [24] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, *et al.*, "Chipyard: Integrated design, simulation, and implementation framework for custom socs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [25] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "Sonicboom: The 3rd generation berkeley out-of-order machine," in *Fourth Workshop on Computer Architecture Research with RISC-V*, vol. 5, pp. 1–7, 2020.
- [26] A. I. Cowen-Rivers, W. Lyu, R. Tutunov, Z. Wang, A. Grosnit, R. R. Griffiths, A. M. Maraval, H. Jianye, J. Wang, J. Peters, *et al.*, "Hebo: Pushing the limits of sample-efficient hyper-parameter optimisation," *Journal of Artificial Intelligence Research*, vol. 74, pp. 1269–1349, 2022.
- [27] "Risc-v unit tests benchmark suites," GitHub, <https://github.com/riscv-software-src/riscv-tests>, 2023.
- [28] C. Wissler, "The spearman correlation formula," *Science*, vol. 22, no. 558, pp. 309–311, 1905.
- [29] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022.
- [30] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. De Freitas, "Taking the human out of the loop: A review of bayesian optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2015.
- [31] S. Daulton, M. Balandat, and E. Bakshy, "Differentiable expected hypervolume improvement for parallel multi-objective bayesian optimization," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9851–9864, 2020.
- [32] M. Lindauer, K. Eggenberger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "Smac3: A versatile bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.
- [33] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyperparameter optimization," *Advances in neural information processing systems*, vol. 24, 2011.
- [34] Y. Ozaki, Y. Tanigaki, S. Watanabe, M. Nomura, and M. Onishi, "Multiobjective tree-structured parzen estimator," *Journal of Artificial Intelligence Research*, vol. 73, pp. 1209–1250, 2022.
- [35] S. Krishnan, A. Yazdanbakhsh, S. Prakash, J. Jabbour, I. Uchendu, S. Ghosh, B. Boroujerdian, D. Richins, D. Tripathy, A. Faust, *et al.*, "Archgym: An open-source gymnasium for machine learning assisted architecture design," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pp. 1–16, 2023.
- [36] T. Liu, N. Astorga, N. Seedat, and M. van der Schaar, "Large language models to enhance bayesian optimization," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.